

#25MML0014

#ANUSREE V

Car Number Plate Detection

```
import tensorflow as tf

from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

import os

folder_path = '/content/drive/MyDrive/numberplate/number plate'

print(f"Checking contents of: {folder_path}")
if os.path.exists(folder_path):
    contents = os.listdir(folder_path)
    if not contents:
        print(f"The directory '{folder_path}' is empty.")
    else:
        print(f"Contents of '{folder_path}': {contents}")
else:
    print(f"Error: The directory '{folder_path}' does not exist.
Please ensure your Google Drive is mounted and the folder path is
correct.")

Checking contents of: /content/drive/MyDrive/numberplate/number plate
Contents of '/content/drive/MyDrive/numberplate/number plate':
['annotations', 'images']

import os
from PIL import Image
import matplotlib.pyplot as plt

image_folder_path = os.path.join(folder_path, 'images')

print(f"Checking contents of: {image_folder_path}")
if os.path.exists(image_folder_path):
    image_files = [f for f in os.listdir(image_folder_path) if
f.lower().endswith(('.png', '.jpg', '.jpeg', '.gif', '.bmp'))]

    if image_files:
        selected_image_name = image_files[0]
        selected_image_path = os.path.join(image_folder_path,
selected_image_name)
```

```

        print(f"Found image: {selected_image_name}. Attempting to
visualize...")
        try:
            img = Image.open(selected_image_path)
            plt.imshow(img)
            plt.title(selected_image_name)
            plt.axis('off')
            plt.show()
        except Exception as e:
            print(f"Error opening or displaying image
{selected_image_name}: {e}")
        else:
            print(f"No image files found in '{image_folder_path}' with
common extensions (.png, .jpg, etc.).")
    else:
        print(f"Error: The image directory '{image_folder_path}' does not
exist. Please check the path.")

```

Checking contents of: /content/drive/MyDrive/numberplate/number
plate/images
Found image: Cars16.png. Attempting to visualize...

Cars16.png



```

image_folder_path = "/content/drive/MyDrive/numberplate/number
plate/images"

```

```

dataset = tf.keras.utils.image_dataset_from_directory(
    image_folder_path,
    labels=None,
    image_size=(224, 224),

```

```

        batch_size=32,
        shuffle=True
    )
Found 433 files.

def preprocess_image(image, target_size):
    image = tf.image.resize(image, target_size)
    image = tf.cast(image, tf.float32) / 255.0
    return image

resized_dataset = dataset.map(preprocess_image)

print("Original dataset element spec:", dataset.element_spec)
print("Processed dataset element spec:", resized_dataset.element_spec)

for images_batch in resized_dataset.take(1):
    print("Shape of one batch:", images_batch.shape)

Original dataset element spec: TensorSpec(shape=(None, 224, 224, 3),
dtype=tf.float32, name=None)
Processed dataset element spec: TensorSpec(shape=(None, 224, 224, 3),
dtype=tf.float32, name=None)
Shape of one batch: (32, 224, 224, 3)

import tensorflow as tf

dataset = tf.keras.utils.image_dataset_from_directory(
    image_folder_path,
    labels=None,
    image_size=(224, 224),
    batch_size=32,
    shuffle=False
)

Found 433 files.

```

Task

Process the `annotations_df` to extract and normalize bounding box coordinates (xmin, ymin, xmax, ymax) for each image, using the `width` and `height` columns for normalization, and then create a TensorFlow dataset that maps image filenames from `"/content/drive/MyDrive/numberplate/number plate/images"` to these normalized bounding box labels, ensuring the bounding box coordinates are normalized relative to the `TARGET_IMAGE_SIZE (224, 224)`.

Process Annotations and Create Dataset

Subtask:

Process the `annotations_df` to extract and normalize bounding box coordinates (xmin, ymin, xmax, ymax) for each image. Then, create a TensorFlow dataset that maps image filenames to these normalized bounding box labels.

Reasoning: I need to implement the steps to process the `annotations_df`, normalize bounding box coordinates, and create a TensorFlow dataset. This involves defining a normalization function, iterating through the annotations, applying normalization, and then constructing a `tf.data.Dataset` using `tf.ragged.constant` for the bounding boxes.

```
import tensorflow as tf
import os

def normalize_bounding_box(box_coords, original_width,
original_height):
    xmin_norm = box_coords['xmin'] / original_width
    ymin_norm = box_coords['ymin'] / original_height
    xmax_norm = box_coords['xmax'] / original_width
    ymax_norm = box_coords['ymax'] / original_height
    return [xmin_norm, ymin_norm, xmax_norm, ymax_norm]

dataset_items = []

for index, row in annotations_df.iterrows():
    filename = row['filename']
    width = row['width']
    height = row['height']
    boxes = row['boxes']

    image_path = os.path.join(image_folder_path, filename)

    normalized_boxes_for_image = []
    for box_dict in boxes:
        normalized_coords = normalize_bounding_box(box_dict, width,
height)
        normalized_boxes_for_image.append(normalized_coords)

    dataset_items.append((image_path, normalized_boxes_for_image))

image_paths = [item[0] for item in dataset_items]
all_normalized_boxes = [item[1] for item in dataset_items]

normalized_bboxes_ragged = tf.ragged.constant(all_normalized_boxes,
dtype=tf.float32)
```

```

bbox_dataset = tf.data.Dataset.from_tensor_slices((image_paths,
normalized_bboxes_ragged))

print("Element spec of bbox_dataset:")
print(bbox_dataset.element_spec)

Element spec of bbox_dataset:
(TensorSpec(shape=(), dtype=tf.string, name=None),
RaggedTensorSpec(TensorShape([None, None]), tf.float32, 1, tf.int64))

```

Load Images and Bounding Boxes

Subtask:

Define a function to load and preprocess images (resize to 224x224 and normalize pixel values) and pair them with their corresponding normalized bounding box coordinates. This function will be applied to the dataset created in the previous step.

Reasoning: I need to define a Python function `load_and_preprocess_image` that takes an image path and normalized bounding boxes, then reads, decodes, resizes, and normalizes the image before returning it along with the bounding boxes. Finally, I will apply this function to the `bbox_dataset` and print the `element_spec` of the resulting `processed_dataset`.

```

def load_and_preprocess_image(image_path, normalized_bboxes):

    img = tf.io.read_file(image_path)

    img = tf.image.decode_image(img, channels=3,
expand_animations=False)

    img = tf.image.resize(img, TARGET_IMAGE_SIZE)

    img = tf.cast(img, tf.float32) / 255.0
    return img, normalized_bboxes

processed_dataset = bbox_dataset.map(load_and_preprocess_image)

print("Element spec of processed_dataset:")
print(processed_dataset.element_spec)

Element spec of processed_dataset:
(TensorSpec(shape=(224, 224, 3), dtype=tf.float32, name=None),
RaggedTensorSpec(TensorShape([None, None]), tf.float32, 1, tf.int64))

```

Split Data and Prepare for Training

Subtask:

Split the prepared dataset into training and validation sets. Ensure the datasets are shuffled, batched, and prefetched for optimal training performance.

Reasoning: I need to split the `processed_dataset` into training and validation sets, applying shuffling, batching, and prefetching as per the instructions. I will first determine the total number of elements, then calculate the split, shuffle the dataset, and finally apply `take`, `skip`, `batch`, and `prefetch` operations.

```
DATASET_SIZE =
tf.data.experimental.cardinality(processed_dataset).numpy()
if DATASET_SIZE == tf.data.experimental.UNKNOWN_CARDINALITY:

    DATASET_SIZE = 0
    for _ in processed_dataset:
        DATASET_SIZE += 1

TRAIN_SPLIT_RATIO = 0.8
VAL_SPLIT_RATIO = 0.2

TRAIN_SIZE = int(TRAIN_SPLIT_RATIO * DATASET_SIZE)
VAL_SIZE = int(VAL_SPLIT_RATIO * DATASET_SIZE)

print(f"Total dataset size: {DATASET_SIZE}")
print(f"Training set size: {TRAIN_SIZE}")
print(f"Validation set size: {VAL_SIZE}")

shuffled_dataset = processed_dataset.shuffle(SHUFFLE_BUFFER_SIZE)

train_dataset = shuffled_dataset.take(TRAIN_SIZE)
val_dataset = shuffled_dataset.skip(TRAIN_SIZE).take(VAL_SIZE)

train_dataset =
train_dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
val_dataset = val_dataset.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print("Train dataset element spec:", train_dataset.element_spec)
print("Validation dataset element spec:", val_dataset.element_spec)

for element in train_dataset.take(1):
    print("Shape of images in one train batch:", element[0].shape)
    print("Shape of bboxes in one train batch:", element[1].shape)

for element in val_dataset.take(1):
```

```

    print("Shape of images in one validation batch:",
          element[0].shape)
    print("Shape of bboxes in one validation batch:",
          element[1].shape)

Total dataset size: 433
Training set size: 346
Validation set size: 86
Train dataset element spec: (TensorSpec(shape=(None, 224, 224, 3),
dtype=tf.float32, name=None), RaggedTensorSpec(TensorShape([None,
None, None]), tf.float32, 2, tf.int64))
Validation dataset element spec: (TensorSpec(shape=(None, 224, 224,
3), dtype=tf.float32, name=None), RaggedTensorSpec(TensorShape([None,
None, None]), tf.float32, 2, tf.int64))
Shape of images in one train batch: (32, 224, 224, 3)
Shape of bboxes in one train batch: (32, None, None)
Shape of images in one validation batch: (32, 224, 224, 3)
Shape of bboxes in one validation batch: (32, None, None)

```

Define CNN Model

Subtask:

Design a Convolutional Neural Network (CNN) model suitable for bounding box regression. This will likely involve using a pre-trained backbone (e.g., MobileNetV2) for feature extraction, followed by custom dense layers that output four values representing the normalized bounding box coordinates (xmin, ymin, xmax, ymax).

Reasoning: I need to define the CNN model for bounding box regression according to the instructions, which includes importing necessary modules, defining the input shape, loading and freezing the MobileNetV2 backbone, building a custom head with dense layers, and finally creating the Keras Model.

```

from tensorflow.keras.layers import Input, GlobalAveragePooling2D,
Dense
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model

input_shape = (*TARGET_IMAGE_SIZE, 3)

base_model = MobileNetV2(input_shape=input_shape,
                          include_top=False,
                          weights='imagenet')

base_model.trainable = False

```

```

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
x = Dense(64, activation='relu')(x)

bbox_output = Dense(4, activation='sigmoid', name='bbox_output')(x)

model = Model(inputs=base_model.input, outputs=bbox_output)

model.summary()

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
 9406464/9406464 ————— 1s 0us/step

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D) input_layer_1[0]...	(None, 112, 112, 32)	864	
bn_Conv1 (BatchNormalizatio...	(None, 112, 112, 32)	128	Conv1[0][0]
Conv1_relu (ReLU) [0]	(None, 112, 112, 32)	0	bn_Conv1[0]
expanded_conv_dept...	(None, 112, 112, 288)	288	Conv1_relu[0]

[0]	(DepthwiseConv2D)	32		
	expanded_conv_dept...	(None, 112, 112,	128	
expanded_conv_de...	(BatchNormalizatio...	32)		
	expanded_conv_dept...	(None, 112, 112,	0	
expanded_conv_de...	(ReLU)	32)		
	expanded_conv_proj...	(None, 112, 112,	512	
expanded_conv_de...	(Conv2D)	16)		
	expanded_conv_proj...	(None, 112, 112,	64	
expanded_conv_pr...	(BatchNormalizatio...	16)		
	block_1_expand	(None, 112, 112,	1,536	
expanded_conv_pr...	(Conv2D)	96)		
	block_1_expand_BN	(None, 112, 112,	384	
block_1_expand[0...	(BatchNormalizatio...	96)		
	block_1_expand_relu	(None, 112, 112,	0	
block_1_expand_B...	(ReLU)	96)		
	block_1_pad	(None, 113, 113,	0	
block_1_expand_r...				

(ZeroPadding2D)	96)		
block_1_depthwise block_1_pad[0][0]	(None, 56, 56, (DepthwiseConv2D)	864	
block_1_depthwise_... block_1_depthwis...	(None, 56, 56, (BatchNormalizatio...	384	
block_1_depthwise_... block_1_depthwis...	(None, 56, 56, (ReLU)	0	
block_1_project block_1_depthwis...	(None, 56, 56, (Conv2D)	2,304	
block_1_project_BN block_1_project[...	(None, 56, 56, (BatchNormalizatio...	96	
block_2_expand block_1_project_...	(None, 56, 56, (Conv2D)	3,456	
block_2_expand_BN block_2_expand[0...	(None, 56, 56, (BatchNormalizatio...	576	
block_2_expand_relu block_2_expand_B...	(None, 56, 56, (ReLU)	0	

block_2_depthwise block_2_expand_r... (DepthwiseConv2D)	(None, 56, 56, 144)	1,296	
block_2_depthwise_... block_2_depthwis... (BatchNormalizatio...	(None, 56, 56, 144)	576	
block_2_depthwise_... block_2_depthwis... (ReLU)	(None, 56, 56, 144)	0	
block_2_project block_2_depthwis... (Conv2D)	(None, 56, 56, 24)	3,456	
block_2_project_BN block_2_project[... (BatchNormalizatio...	(None, 56, 56, 24)	96	
block_2_add (Add) block_1_project_... block_2_project_...	(None, 56, 56, 24)	0	
block_3_expand block_2_add[0][0] (Conv2D)	(None, 56, 56, 144)	3,456	
block_3_expand_BN block_3_expand[0... (BatchNormalizatio...	(None, 56, 56, 144)	576	

block_3_expand_relu	(None, 56, 56,	0	
block_3_expand_B...	(ReLU)	144)	
block_3_pad	(None, 57, 57,	0	
block_3_expand_r...	(ZeroPadding2D)	144)	
block_3_depthwise	(None, 28, 28,	1,296	
block_3_pad[0][0]	(DepthwiseConv2D)	144)	
block_3_depthwise_...	(None, 28, 28,	576	
block_3_depthwis...	(BatchNormalizatio...	144)	
block_3_depthwise_...	(None, 28, 28,	0	
block_3_depthwis...	(ReLU)	144)	
block_3_project	(None, 28, 28,	4,608	
block_3_depthwis...	(Conv2D)	32)	
block_3_project_BN	(None, 28, 28,	128	
block_3_project[...	(BatchNormalizatio...	32)	
block_4_expand	(None, 28, 28,	6,144	
block_3_project_...	(Conv2D)	192)	

block_4_expand_BN	(None, 28, 28,	768
block_4_expand[0...	(BatchNormalizatio...	192)

block_4_expand_relu	(None, 28, 28,	0
block_4_expand_B...	(ReLU)	192)

block_4_depthwise	(None, 28, 28,	1,728
block_4_expand_r...	(DepthwiseConv2D)	192)

block_4_depthwise_...	(None, 28, 28,	768
block_4_depthwis...	(BatchNormalizatio...	192)

block_4_depthwise_...	(None, 28, 28,	0
block_4_depthwis...	(ReLU)	192)

block_4_project	(None, 28, 28,	6,144
block_4_depthwis...	(Conv2D)	32)

block_4_project_BN	(None, 28, 28,	128
block_4_project[...	(BatchNormalizatio...	32)

block_4_add (Add)	(None, 28, 28,	0
block_3_project_...	32)	
block_4_project_...		

block_5_expand block_4_add[0][0]	(None, 28, 28,	6,144
(Conv2D)	192)	
block_5_expand_BN block_5_expand[0...	(None, 28, 28,	768
(BatchNormalizatio...	192)	
block_5_expand_relu block_5_expand_B...	(None, 28, 28,	0
(ReLU)	192)	
block_5_depthwise block_5_expand_r...	(None, 28, 28,	1,728
(DepthwiseConv2D)	192)	
block_5_depthwise_... block_5_depthwis...	(None, 28, 28,	768
(BatchNormalizatio...	192)	
block_5_depthwise_... block_5_depthwis...	(None, 28, 28,	0
(ReLU)	192)	
block_5_project block_5_depthwis...	(None, 28, 28,	6,144
(Conv2D)	32)	
block_5_project_BN block_5_project[...	(None, 28, 28,	128
(BatchNormalizatio...	32)	
block_5_add (Add)	(None, 28, 28,	0

block_4_add[0][0]	32)		
block_5_project...			
block_6_expand	(None, 28, 28,	6,144	
block_5_add[0][0]	(Conv2D)	192)	
block_6_expand_BN	(None, 28, 28,	768	
block_6_expand[0]	(BatchNormalizatio...	192)	
block_6_expand_relu	(None, 28, 28,	0	
block_6_expand_B...	(ReLU)	192)	
block_6_pad	(None, 29, 29,	0	
block_6_expand_r...	(ZeroPadding2D)	192)	
block_6_depthwise	(None, 14, 14,	1,728	
block_6_pad[0][0]	(DepthwiseConv2D)	192)	
block_6_depthwise_...	(None, 14, 14,	768	
block_6_depthwis...	(BatchNormalizatio...	192)	
block_6_depthwise_...	(None, 14, 14,	0	
block_6_depthwis...	(ReLU)	192)	
block_6_project	(None, 14, 14,	12,288	
block_6_depthwis...			

(Conv2D)	64)		
block_6_project_BN	(None, 14, 14,	256	
block_6_project[...]	(BatchNormalizatio...	64)	
block_7_expand	(None, 14, 14,	24,576	
block_6_project_...	(Conv2D)	384)	
block_7_expand_BN	(None, 14, 14,	1,536	
block_7_expand[0...]	(BatchNormalizatio...	384)	
block_7_expand_relu	(None, 14, 14,	0	
block_7_expand_B...	(ReLU)	384)	
block_7_depthwise	(None, 14, 14,	3,456	
block_7_expand_r...	(DepthwiseConv2D)	384)	
block_7_depthwise_...	(None, 14, 14,	1,536	
block_7_depthwis...	(BatchNormalizatio...	384)	
block_7_depthwise_...	(None, 14, 14,	0	
block_7_depthwis...	(ReLU)	384)	
block_7_project	(None, 14, 14,	24,576	
block_7_depthwis...	(Conv2D)	64)	

block_7_project_BN	(None, 14, 14,	256	
block_7_project[...] (BatchNormalizatio...	64)		
block_7_add (Add)	(None, 14, 14,	0	
block_6_project_...	64)		
block_7_project_...			
block_8_expand	(None, 14, 14,	24,576	
block_7_add[0][0] (Conv2D)	384)		
block_8_expand_BN	(None, 14, 14,	1,536	
block_8_expand[0...] (BatchNormalizatio...	384)		
block_8_expand_relu	(None, 14, 14,	0	
block_8_expand_B... (ReLU)	384)		
block_8_depthwise	(None, 14, 14,	3,456	
block_8_expand_r... (DepthwiseConv2D)	384)		
block_8_depthwise_...	(None, 14, 14,	1,536	
block_8_depthwis... (BatchNormalizatio...	384)		
block_8_depthwise_...	(None, 14, 14,	0	
block_8_depthwis... (ReLU)	384)		

block_8_project	(None, 14, 14,	24,576	
block_8_depthwis...	(Conv2D)	64)	
block_8_project_BN	(None, 14, 14,	256	
block_8_project[...	(BatchNormalizatio...	64)	
block_8_add (Add)	(None, 14, 14,	0	
block_7_add[0][0...			
	64)		
block_8_project_...			
block_9_expand	(None, 14, 14,	24,576	
block_8_add[0][0]	(Conv2D)	384)	
block_9_expand_BN	(None, 14, 14,	1,536	
block_9_expand[0...	(BatchNormalizatio...	384)	
block_9_expand_relu	(None, 14, 14,	0	
block_9_expand_B...	(ReLU)	384)	
block_9_depthwise	(None, 14, 14,	3,456	
block_9_expand_r...	(DepthwiseConv2D)	384)	
block_9_depthwise_...	(None, 14, 14,	1,536	
block_9_depthwis...	(BatchNormalizatio...	384)	

block_9_depthwise_...	(None, 14, 14,	0
block_9_depthwis...	(ReLU)	384)
block_9_project	(None, 14, 14,	24,576
block_9_depthwis...	(Conv2D)	64)
block_9_project_BN	(None, 14, 14,	256
block_9_project[...	(BatchNormalizatio...	64)
block_9_add (Add)	(None, 14, 14,	0
block_8_add[0][0...	64)	
block_9_project_...		
block_10_expand	(None, 14, 14,	24,576
block_9_add[0][0]	(Conv2D)	384)
block_10_expand_BN	(None, 14, 14,	1,536
block_10_expand[...	(BatchNormalizatio...	384)
block_10_expand_re...	(None, 14, 14,	0
block_10_expand_...	(ReLU)	384)
block_10_depthwise	(None, 14, 14,	3,456
block_10_expand_...	(DepthwiseConv2D)	384)
block_10_depthwise...	(None, 14, 14,	1,536

block_10_depthwi...	(BatchNormalizatio...	384)		
block_10_depthwise...	(None, 14, 14,		0	
block_10_depthwi...	(ReLU)	384)		
block_10_project	(None, 14, 14,		36,864	
block_10_depthwi...	(Conv2D)	96)		
block_10_project_BN	(None, 14, 14,		384	
block_10_project...	(BatchNormalizatio...	96)		
block_11_expand	(None, 14, 14,		55,296	
block_10_project...	(Conv2D)	576)		
block_11_expand_BN	(None, 14, 14,		2,304	
block_11_expand[...	(BatchNormalizatio...	576)		
block_11_expand_re...	(None, 14, 14,		0	
block_11_expand_...	(ReLU)	576)		
block_11_depthwise	(None, 14, 14,		5,184	
block_11_expand_...	(DepthwiseConv2D)	576)		
block_11_depthwise...	(None, 14, 14,		2,304	
block_11_depthwi...				

(BatchNormalization)	576)		
block_11_depthwise	(None, 14, 14,	0	
block_11_depthwi	(ReLU)	576)	
block_11_project	(None, 14, 14,	55,296	
block_11_depthwi	(Conv2D)	96)	
block_11_project_BN	(None, 14, 14,	384	
block_11_project	(BatchNormalization)	96)	
block_11_add (Add)	(None, 14, 14,	0	
block_10_project		96)	
block_11_project			
block_12_expand	(None, 14, 14,	55,296	
block_11_add[0]	(Conv2D)	576)	
block_12_expand_BN	(None, 14, 14,	2,304	
block_12_expand	(BatchNormalization)	576)	
block_12_expand_re	(None, 14, 14,	0	
block_12_expand	(ReLU)	576)	
block_12_depthwise	(None, 14, 14,	5,184	
block_12_expand	(DepthwiseConv2D)	576)	

block_12_depthwise...	(None, 14, 14,	2,304	
block_12_depthwi...	(BatchNormalizatio...	576)	
block_12_depthwise...	(None, 14, 14,	0	
block_12_depthwi...	(ReLU)	576)	
block_12_project	(None, 14, 14,	55,296	
block_12_depthwi...	(Conv2D)	96)	
block_12_project_BN	(None, 14, 14,	384	
block_12_project...	(BatchNormalizatio...	96)	
block_12_add (Add)	(None, 14, 14,	0	
block_11_add[0][...		96)	
block_12_project...			
block_13_expand	(None, 14, 14,	55,296	
block_12_add[0][...	(Conv2D)	576)	
block_13_expand_BN	(None, 14, 14,	2,304	
block_13_expand[...	(BatchNormalizatio...	576)	
block_13_expand_re...	(None, 14, 14,	0	
block_13_expand_...	(ReLU)	576)	

block_13_pad	(None, 15, 15,	0	
block_13_expand_...	(ZeroPadding2D)	576)	
block_13_depthwise	(None, 7, 7, 576)	5,184	
block_13_pad[0][...	(DepthwiseConv2D)		
block_13_depthwise...	(None, 7, 7, 576)	2,304	
block_13_depthwi...	(BatchNormalizatio...		
block_13_depthwise...	(None, 7, 7, 576)	0	
block_13_depthwi...	(ReLU)		
block_13_project	(None, 7, 7, 160)	92,160	
block_13_depthwi...	(Conv2D)		
block_13_project_BN	(None, 7, 7, 160)	640	
block_13_project...	(BatchNormalizatio...		
block_14_expand	(None, 7, 7, 960)	153,600	
block_13_project...	(Conv2D)		
block_14_expand_BN	(None, 7, 7, 960)	3,840	
block_14_expand[...	(BatchNormalizatio...		

block_14_expand_re...	(None, 7, 7, 960)	0	
block_14_expand_... (ReLU)			
block_14_depthwise	(None, 7, 7, 960)	8,640	
block_14_expand_... (DepthwiseConv2D)			
block_14_depthwise...	(None, 7, 7, 960)	3,840	
block_14_depthwi... (BatchNormalizatio...			
block_14_depthwise...	(None, 7, 7, 960)	0	
block_14_depthwi... (ReLU)			
block_14_project	(None, 7, 7, 160)	153,600	
block_14_depthwi... (Conv2D)			
block_14_project_BN	(None, 7, 7, 160)	640	
block_14_project... (BatchNormalizatio...			
block_14_add (Add)	(None, 7, 7, 160)	0	
block_13_project...			
block_14_project...			
block_15_expand	(None, 7, 7, 960)	153,600	
block_14_add[0][... (Conv2D)			

block_15_expand_BN block_15_expand[...	(None, 7, 7, 960)	3,840	
(BatchNormalizatio...			
block_15_expand_re... block_15_expand_...	(None, 7, 7, 960)	0	
(ReLU)			
block_15_depthwise block_15_expand_...	(None, 7, 7, 960)	8,640	
(DepthwiseConv2D)			
block_15_depthwise... block_15_depthwi...	(None, 7, 7, 960)	3,840	
(BatchNormalizatio...			
block_15_depthwise... block_15_depthwi...	(None, 7, 7, 960)	0	
(ReLU)			
block_15_project block_15_depthwi...	(None, 7, 7, 160)	153,600	
(Conv2D)			
block_15_project_BN block_15_project...	(None, 7, 7, 160)	640	
(BatchNormalizatio...			
block_15_add (Add) block_14_add[0][...	(None, 7, 7, 160)	0	
block_15_project...			
block_16_expand	(None, 7, 7, 960)	153,600	

block_15_add[0][... (Conv2D)			
block_16_expand_BN block_16_expand[... (BatchNormalizatio...	(None, 7, 7, 960)	3,840	
block_16_expand_re... block_16_expand_... (ReLU)	(None, 7, 7, 960)	0	
block_16_depthwise block_16_expand_... (DepthwiseConv2D)	(None, 7, 7, 960)	8,640	
block_16_depthwise... block_16_depthwi... (BatchNormalizatio...	(None, 7, 7, 960)	3,840	
block_16_depthwise... block_16_depthwi... (ReLU)	(None, 7, 7, 960)	0	
block_16_project block_16_depthwi... (Conv2D)	(None, 7, 7, 320)	307,200	
block_16_project_BN block_16_project... (BatchNormalizatio...	(None, 7, 7, 320)	1,280	
Conv_1 (Conv2D) block_16_project...	(None, 7, 7,	409,600	

	1280)		
Conv_1_bn	(None, 7, 7,	5,120	Conv_1[0][0]
(BatchNormalizatio...	1280)		
out_relu (ReLU)	(None, 7, 7,	0	Conv_1_bn[0]
[0]	1280)		
global_average_poo...	(None, 1280)	0	out_relu[0]
[0]	(GlobalAveragePool...		
dense (Dense)	(None, 128)	163,968	
global_average_p...			
dense_1 (Dense)	(None, 64)	8,256	dense[0][0]
bbox_output (Dense)	(None, 4)	260	dense_1[0][0]
Total params: 2,430,468 (9.27 MB)			
Trainable params: 172,484 (673.77 KB)			
Non-trainable params: 2,257,984 (8.61 MB)			

Define Custom Loss Function

Subtask:

Define a custom loss function for bounding box regression. This loss function should be able to handle variable numbers of bounding boxes per image (ragged tensors) and calculate the mean squared error for each bounding box prediction, averaging the loss across all boxes in a batch. If an image has no bounding boxes, its loss should be 0.

Instructions

1. Import the necessary modules from TensorFlow: `tf`.
2. Define a function `bbox_loss` that takes `y_true` (ground truth bounding boxes) and `y_pred` (predicted bounding boxes) as input.
3. Convert `y_true` to a dense tensor for easier manipulation, handling cases where there are no bounding boxes.
4. Calculate the mean squared error between `y_true` and `y_pred`.
5. Handle the varying number of bounding boxes per image by ensuring that the loss is only calculated for existing boxes and correctly averaged. For images with no bounding boxes, the loss contribution should be zero.
6. Return the computed loss.

Reasoning: I need to define the custom loss function as described in the subtask instructions. This involves handling ragged tensors, calculating MSE only for valid bounding boxes, and ensuring that images without bounding boxes contribute zero loss.

```
import tensorflow as tf

def bbox_loss(y_true, y_pred):
    if not isinstance(y_true, tf.RaggedTensor):
        mask = tf.reduce_any(tf.not_equal(y_true, -1.0), axis=-1)
        row_lengths = tf.reduce_sum(tf.cast(mask, tf.int64), axis=1)
        y_true =
tf.RaggedTensor.from_row_lengths(tf.boolean_mask(y_true, mask),
row_lengths)

    batch_losses = []
    for i in tf.range(tf.shape(y_pred)[0]):
        true_boxes_for_image = y_true[i]
        predicted_box_for_image = y_pred[i]

        if tf.equal(tf.shape(true_boxes_for_image)[0], 0):
            batch_losses.append(0.0)
        else:
            predicted_boxes_repeated =
tf.tile(tf.expand_dims(predicted_box_for_image, axis=0),
[tf.shape(true_boxes_for_image)[0], 1])

            mse = tf.reduce_mean(tf.square(true_boxes_for_image -
predicted_boxes_repeated))
            batch_losses.append(mse)

    return tf.reduce_mean(tf.stack(batch_losses))

print("Custom bounding box loss function 'bbox_loss' defined.")
```

Custom bounding box loss function 'bbox_loss' defined.

Reasoning: The model and custom loss function have been defined. The next logical step is to compile the model using an appropriate optimizer and the custom loss function to prepare it for training.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss=bbox_loss)

print("Model compiled successfully with Adam optimizer and custom
bbox_loss.")
```

Model compiled successfully with Adam optimizer and custom bbox_loss.

Compile and Train Model

Subtask:

Train the model on the training dataset and evaluate it using the validation dataset.

Reasoning: I need to train the model using the `fit` method with the specified training and validation datasets, and a given number of epochs.

Reasoning: The `OperatorNotAllowedInGraphError` indicates that the custom loss function `bbox_loss` is iterating over symbolic TensorFlow tensors, which is not permitted in graph mode. Decorating the function with `@tf.function` will enable AutoGraph to convert the Python logic into a static TensorFlow graph, resolving this issue.

```
import tensorflow as tf

@tf.function
def bbox_loss(y_true, y_pred):
    if not isinstance(y_true, tf.RaggedTensor):
        pass

    batch_losses = []
    for i in tf.range(tf.shape(y_pred)[0]):
        true_boxes_for_image = y_true[i]
        predicted_box_for_image = y_pred[i]

        if tf.equal(tf.shape(true_boxes_for_image)[0], 0):
            batch_losses.append(0.0)
        else:
```

```

        predicted_boxes_repeated =
tf.tile(tf.expand_dims(predicted_box_for_image, axis=0),
[tf.shape(true_boxes_for_image)[0], 1])

        mse = tf.reduce_mean(tf.square(true_boxes_for_image -
predicted_boxes_repeated))
        batch_losses.append(mse)

    return tf.reduce_mean(tf.stack(batch_losses))

print("Custom bounding box loss function 'bbox_loss' defined and
decorated with @tf.function.")

```

Custom bounding box loss function 'bbox_loss' defined and decorated with @tf.function.

Reasoning: The custom loss function `bbox_loss` has been redefined with `@tf.function`. Now, the model needs to be recompiled with this updated loss function to ensure the changes are applied before attempting to train it again.

```

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss=bbox_loss)

print("Model compiled successfully with Adam optimizer and custom
bbox_loss.")

```

Model compiled successfully with Adam optimizer and custom `bbox_loss`.

Reasoning: The model has been successfully recompiled with the corrected custom loss function. Now, I will re-attempt to train the model using the `fit` method with the specified training and validation datasets for the given number of epochs.

Reasoning: The `InaccessibleTensorError` indicates that tensors created conditionally inside a loop within a `@tf.function` cannot be accessed outside their scope. To fix this, I will refactor the `bbox_loss` function to use `tf.TensorArray` for accumulating losses and `tf.cond` for conditional logic, ensuring that a consistent tensor type is produced by each branch of the conditional statement, which is compatible with TensorFlow's graph mode.

```

import tensorflow as tf

@tf.function
def bbox_loss(y_true, y_pred):

    batch_size = tf.shape(y_pred)[0]

    losses_ta = tf.TensorArray(dtype=tf.float32, size=batch_size,
dynamic_size=False)

```

```

for i in tf.range(batch_size):
    true_boxes_for_image = y_true[i]
    predicted_box_for_image = y_pred[i]

    num_true_boxes = tf.shape(true_boxes_for_image)[0]

    loss_for_image = tf.cond(
        tf.equal(num_true_boxes, 0),
        lambda: tf.constant(0.0, dtype=tf.float32),
        lambda: tf.reduce_mean(tf.square(true_boxes_for_image -
tf.tile(tf.expand_dims(predicted_box_for_image, axis=0),
[num_true_boxes, 1]))))
    )
    losses_ta = losses_ta.write(i, loss_for_image)

return tf.reduce_mean(losses_ta.stack())

print("Custom bounding box loss function 'bbox_loss' redefined with
tf.TensorArray and tf.cond.")

```

Custom bounding box loss function 'bbox_loss' redefined with
tf.TensorArray and tf.cond.

Reasoning: The custom loss function `bbox_loss` has been redefined with `tf.TensorArray` and `tf.cond` to resolve the `InaccessibleTensorError`. Now, the model needs to be recompiled with this updated loss function to ensure the changes are applied before attempting to train it again.

```

optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=optimizer, loss=bbox_loss)

print("Model compiled successfully with Adam optimizer and custom
bbox_loss.")

```

Model compiled successfully with Adam optimizer and custom `bbox_loss`.

Reasoning: The model has been successfully recompiled with the corrected custom loss function. Now, I will re-attempt to train the model using the `fit` method with the specified training and validation datasets for the given number of epochs.

```

epochs = 50
history = model.fit(
    train_dataset,
    epochs=epochs,
    validation_data=val_dataset
)

print("Model training completed.")

```

```
Epoch 1/50
11/11 _____ 49s 3s/step - loss: 0.0361 - val_loss:
0.0259
Epoch 2/50
11/11 _____ 45s 3s/step - loss: 0.0227 - val_loss:
0.0175
Epoch 3/50
11/11 _____ 34s 3s/step - loss: 0.0189 - val_loss:
0.0158
Epoch 4/50
11/11 _____ 42s 3s/step - loss: 0.0111 - val_loss:
0.0107
Epoch 5/50
11/11 _____ 38s 2s/step - loss: 0.0113 - val_loss:
0.0105
Epoch 6/50
11/11 _____ 36s 3s/step - loss: 0.0094 - val_loss:
0.0064
Epoch 7/50
11/11 _____ 35s 3s/step - loss: 0.0076 - val_loss:
0.0065
Epoch 8/50
11/11 _____ 39s 2s/step - loss: 0.0077 - val_loss:
0.0053
Epoch 9/50
11/11 _____ 36s 3s/step - loss: 0.0049 - val_loss:
0.0041
Epoch 10/50
11/11 _____ 34s 2s/step - loss: 0.0049 - val_loss:
0.0034
Epoch 11/50
11/11 _____ 33s 3s/step - loss: 0.0037 - val_loss:
0.0045
Epoch 12/50
11/11 _____ 42s 3s/step - loss: 0.0033 - val_loss:
0.0038
Epoch 13/50
11/11 _____ 40s 3s/step - loss: 0.0036 - val_loss:
0.0030
Epoch 14/50
11/11 _____ 43s 3s/step - loss: 0.0034 - val_loss:
0.0025
Epoch 15/50
11/11 _____ 35s 3s/step - loss: 0.0027 - val_loss:
0.0034
Epoch 16/50
11/11 _____ 39s 3s/step - loss: 0.0025 - val_loss:
0.0028
Epoch 17/50
11/11 _____ 40s 2s/step - loss: 0.0024 - val_loss:
```



```
9.6897e-04
Epoch 18/50
11/11 _____ 33s 2s/step - loss: 0.0021 - val_loss:
0.0056
Epoch 19/50
11/11 _____ 38s 2s/step - loss: 0.0025 - val_loss:
0.0032
Epoch 20/50
11/11 _____ 33s 2s/step - loss: 0.0033 - val_loss:
0.0028
Epoch 21/50
11/11 _____ 34s 2s/step - loss: 0.0026 - val_loss:
0.0036
Epoch 22/50
11/11 _____ 38s 2s/step - loss: 0.0037 - val_loss:
0.0017
Epoch 23/50
11/11 _____ 31s 2s/step - loss: 0.0030 - val_loss:
0.0035
Epoch 24/50
11/11 _____ 49s 3s/step - loss: 0.0030 - val_loss:
0.0023
Epoch 25/50
11/11 _____ 32s 3s/step - loss: 0.0024 - val_loss:
0.0020
Epoch 26/50
11/11 _____ 42s 3s/step - loss: 0.0028 - val_loss:
0.0024
Epoch 27/50
11/11 _____ 41s 3s/step - loss: 0.0036 - val_loss:
0.0029
Epoch 28/50
11/11 _____ 32s 2s/step - loss: 0.0026 - val_loss:
0.0013
Epoch 29/50
11/11 _____ 35s 3s/step - loss: 0.0035 - val_loss:
0.0037
Epoch 30/50
11/11 _____ 43s 3s/step - loss: 0.0026 - val_loss:
0.0016
Epoch 31/50
11/11 _____ 32s 2s/step - loss: 0.0028 - val_loss:
0.0026
Epoch 32/50
11/11 _____ 39s 2s/step - loss: 0.0026 - val_loss:
0.0016
Epoch 33/50
11/11 _____ 42s 3s/step - loss: 0.0037 - val_loss:
0.0038
Epoch 34/50
```

```
11/11 _____ 34s 3s/step - loss: 0.0029 - val_loss: 0.0015
Epoch 35/50
11/11 _____ 41s 3s/step - loss: 0.0022 - val_loss: 0.0013
Epoch 36/50
11/11 _____ 33s 3s/step - loss: 0.0022 - val_loss: 0.0037
Epoch 37/50
11/11 _____ 42s 3s/step - loss: 0.0038 - val_loss: 0.0054
Epoch 38/50
11/11 _____ 32s 3s/step - loss: 0.0030 - val_loss: 0.0022
Epoch 39/50
11/11 _____ 41s 2s/step - loss: 0.0022 - val_loss: 0.0026
Epoch 40/50
11/11 _____ 51s 3s/step - loss: 0.0031 - val_loss: 0.0017
Epoch 41/50
11/11 _____ 32s 3s/step - loss: 0.0032 - val_loss: 0.0010
Epoch 42/50
11/11 _____ 52s 4s/step - loss: 0.0034 - val_loss: 0.0015
Epoch 43/50
11/11 _____ 34s 3s/step - loss: 0.0016 - val_loss: 0.0027
Epoch 44/50
11/11 _____ 38s 2s/step - loss: 0.0023 - val_loss: 0.0023
Epoch 45/50
11/11 _____ 41s 2s/step - loss: 0.0018 - val_loss: 0.0050
Epoch 46/50
11/11 _____ 32s 2s/step - loss: 0.0028 - val_loss: 0.0017
Epoch 47/50
11/11 _____ 33s 2s/step - loss: 0.0024 - val_loss: 0.0027
Epoch 48/50
11/11 _____ 32s 2s/step - loss: 0.0032 - val_loss: 0.0026
Epoch 49/50
11/11 _____ 32s 2s/step - loss: 0.0029 - val_loss: 0.0017
Epoch 50/50
11/11 _____ 32s 2s/step - loss: 0.0021 - val_loss:
```

```
0.0018
Model training completed.
```

Evaluate and Visualize Predictions

Subtask:

Evaluate the trained model's performance on the validation set. Additionally, visualize the model's predictions by taking a few sample images, running inference, and drawing the predicted bounding boxes on them to qualitatively assess the detection accuracy.

Reasoning: First, I will evaluate the trained model on the validation dataset to get an overall performance metric. This will give a quantitative measure of how well the model is performing on unseen data.

```
val_loss = model.evaluate(val_dataset)
print(f"Validation Loss: {val_loss:.4f}")

3/3 ————— 19s 1s/step - loss: 0.0039
Validation Loss: 0.0035
```

Reasoning: Before visualizing the predictions, I need a helper function to draw bounding boxes on an image. This function will take the image data, bounding box coordinates (normalized), and the target image size to convert the normalized coordinates back to pixel values for drawing.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def draw_boxes(image, boxes, color='red', label=None,
               image_size=TARGET_IMAGE_SIZE):
    fig, ax = plt.subplots(1, figsize=(8, 8))
    ax.imshow(image)

    for box in boxes:
        xmin = box[0] * image_size[1]
        ymin = box[1] * image_size[0]
        xmax = box[2] * image_size[1]
        ymax = box[3] * image_size[0]

        rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax -
                                ymin,
                                linewidth=2, edgecolor=color,
                                facecolor='none')
        ax.add_patch(rect)

    if label:
```

```
ax.set_title(label)
ax.axis('off')
plt.show()
```

```
print("Helper function 'draw_boxes' defined.")
```

Helper function 'draw_boxes' defined.

Reasoning: Now that the `draw_boxes` helper function is defined, I will take a few samples from the `val_dataset`, get model predictions for these samples, and then use the `draw_boxes` function to visualize both the ground truth and predicted bounding boxes on the images. I will display both ground truth and predicted bounding boxes on the same image, using different colors to distinguish them.

```
num_samples_to_visualize = 5
```

```
for i, (images_batch, gt_bboxes_batch) in
    enumerate(val_dataset.take(num_samples_to_visualize)):
    print(f"Visualizing batch {i+1}/{num_samples_to_visualize}")

    predicted_bboxes_batch = model.predict(images_batch)

    for j in range(images_batch.shape[0]):
        if j >= num_samples_to_visualize:
            break

        image = images_batch[j].numpy()

        gt_boxes_for_image = gt_bboxes_batch[j].numpy()

        predicted_box_for_image = predicted_bboxes_batch[j:j+1]

        if predicted_box_for_image.shape == (4,):
            predicted_box_for_image =
np.expand_dims(predicted_box_for_image, axis=0)

        draw_boxes(image, gt_boxes_for_image, color='green',
label=f'Image {i*BATCH_SIZE + j + 1}: Ground Truth BBoxes',
image_size=TARGET_IMAGE_SIZE)

        draw_boxes(image, predicted_box_for_image, color='red',
label=f'Image {i*BATCH_SIZE + j + 1}: Predicted BBox',
image_size=TARGET_IMAGE_SIZE)
```

Visualizing batch 1/5

1/1 ————— 3s 3s/step

Image 1: Ground Truth BBoxes



Image 1: Predicted BBox



Image 2: Ground Truth BBoxes



Image 2: Predicted BBox



Image 3: Ground Truth BBoxes



Image 3: Predicted BBox



Image 4: Ground Truth BBoxes



Image 4: Predicted BBox



Image 5: Ground Truth BBoxes



Image 5: Predicted BBox



Visualizing batch 2/5

1/1 ————— 1s 1s/step

Image 33: Ground Truth BBoxes



Image 33: Predicted BBox



Image 34: Ground Truth BBoxes



Image 34: Predicted BBox



Image 35: Ground Truth BBoxes



Image 35: Predicted BBox



Image 36: Ground Truth BBoxes



Image 36: Predicted BBox



Image 37: Ground Truth BBoxes



Image 37: Predicted BBox



Visualizing batch 3/5

1/1 ————— 3s 3s/step

Image 65: Ground Truth BBoxes



Image 65: Predicted BBox



Image 66: Ground Truth BBoxes



Image 66: Predicted BBox



Image 67: Ground Truth BBoxes

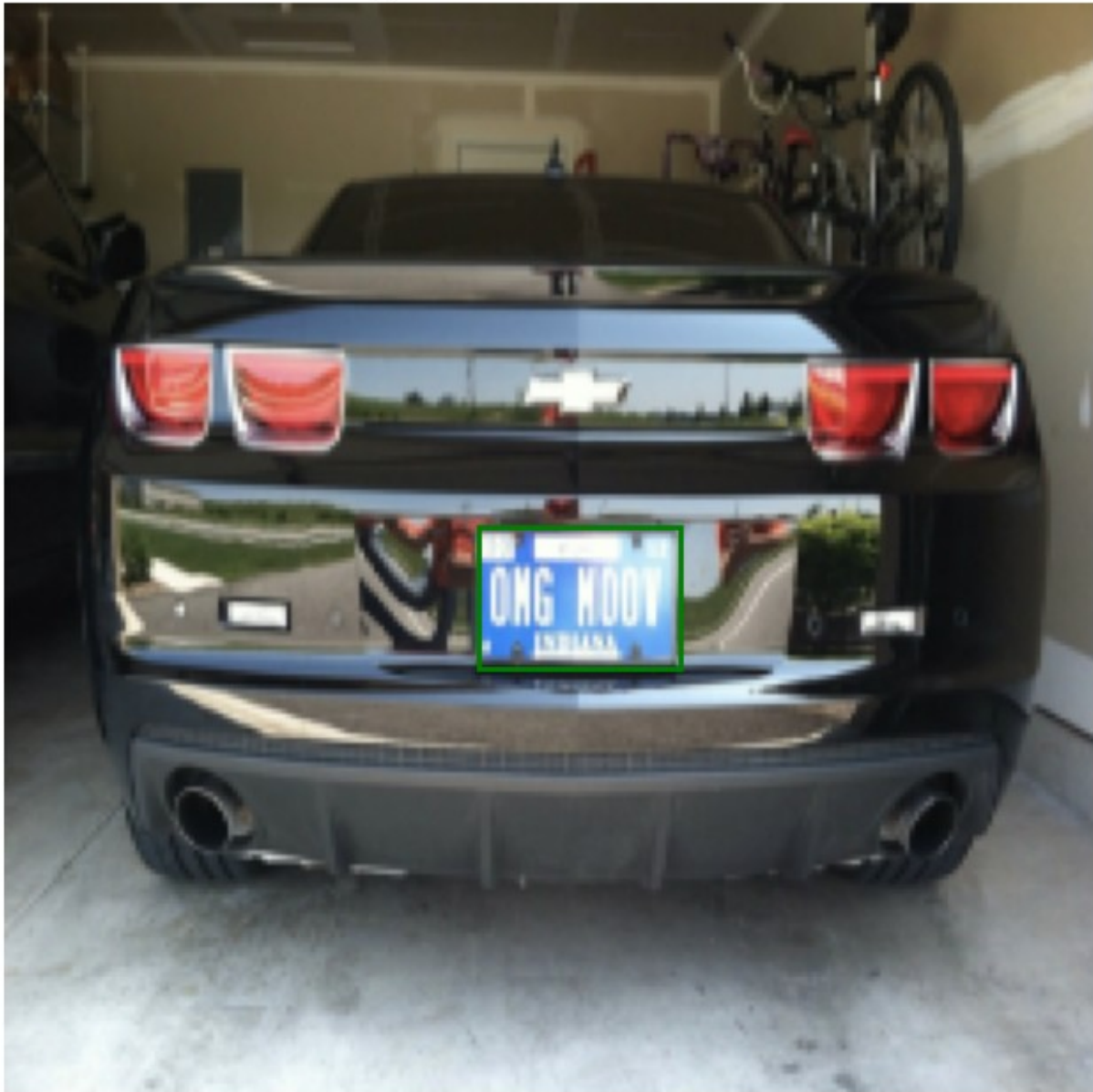


Image 67: Predicted BBox

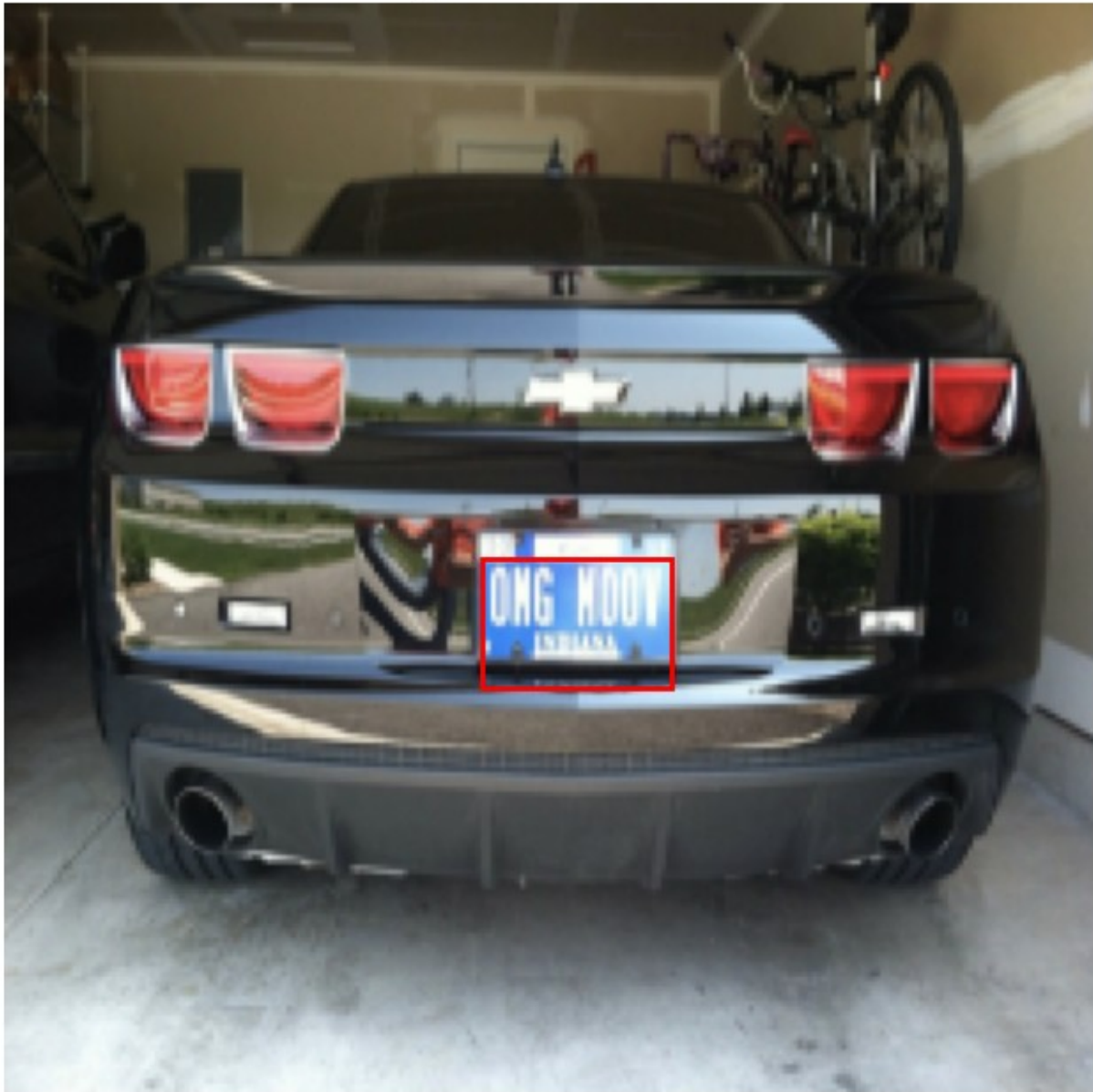


Image 68: Ground Truth BBoxes



Image 68: Predicted BBox



Image 69: Ground Truth BBoxes



Image 69: Predicted BBox



Reasoning: The previous code block visualized ground truth and predicted bounding boxes on separate plots. To align with the instructions to display both on the same image with different colors, I will modify the code to uncomment the combined visualization logic and comment out the separate `draw_boxes` calls.

```
num_samples_to_visualize = 5

for i, (images_batch, gt_bboxes_batch) in
    enumerate(val_dataset.take(num_samples_to_visualize)):
        print(f"Visualizing batch {i+1}/{num_samples_to_visualize}")
```

```

predicted_bboxes_batch = model.predict(images_batch)

for j in range(images_batch.shape[0]):
    if j >= num_samples_to_visualize:

        image = images_batch[j].numpy()

        gt_boxes_for_image = gt_bboxes_batch[j].numpy()

        predicted_box_for_image = predicted_bboxes_batch[j:j+1]

        if predicted_box_for_image.shape == (4,):
            predicted_box_for_image =
np.expand_dims(predicted_box_for_image, axis=0)

        all_boxes = list(gt_boxes_for_image) +
list(predicted_box_for_image)
        all_colors = ['green'] * len(gt_boxes_for_image) + ['red'] *
len(predicted_box_for_image)
        if all_boxes:
            fig, ax = plt.subplots(1, figsize=(8, 8))
            ax.imshow(image)
            for k, box in enumerate(all_boxes):
                xmin, ymin, xmax, ymax = box[0]*TARGET_IMAGE_SIZE[1],
box[1]*TARGET_IMAGE_SIZE[0], box[2]*TARGET_IMAGE_SIZE[1],
box[3]*TARGET_IMAGE_SIZE[0]
                rect = patches.Rectangle((xmin, ymin), xmax - xmin,
ymax - ymin, linewidth=2, edgecolor=all_colors[k], facecolor='none')
                ax.add_patch(rect)
                ax.set_title(f'Image {i*BATCH_SIZE + j + 1}: GT (Green) &
Pred (Red)')
                ax.axis('off')
                plt.show()
        else:
            plt.imshow(image)
            plt.title(f'Image {i*BATCH_SIZE + j + 1}: No BBoxes')
            ax.axis('off')
            plt.show()

```

Visualizing batch 1/5

1/1  1s 1s/step

Image 1: GT (Green) & Pred (Red)



Image 2: GT (Green) & Pred (Red)



Image 3: GT (Green) & Pred (Red)



Image 4: GT (Green) & Pred (Red)



Image 5: GT (Green) & Pred (Red)



Visualizing batch 2/5

1/1 ————— 1s 1s/step

Image 33: GT (Green) & Pred (Red)



Image 34: GT (Green) & Pred (Red)



Image 35: GT (Green) & Pred (Red)



Image 36: GT (Green) & Pred (Red)



Image 37: GT (Green) & Pred (Red)



Visualizing batch 3/5

1/1 ————— 1s 1s/step

Image 65: GT (Green) & Pred (Red)



Image 66: GT (Green) & Pred (Red)



Image 67: GT (Green) & Pred (Red)



Image 68: GT (Green) & Pred (Red)



Image 69: GT (Green) & Pred (Red)



Reasoning: The previous visualization attempt might have had issues with `plt.show()` in a loop and redundant `num_samples_to_visualize` logic. To ensure all plots are displayed correctly and to visualize the specified number of samples accurately, I will first extract individual samples from the validation dataset, then iterate through these samples to make predictions and display both ground truth and predicted bounding boxes on the same image.

```
num_samples_to_visualize = 5
```

```
sample_images = []  
sample_gt_bboxes = []
```

```

for image_batch, gt_bbox_batch in
val_dataset.unbatch().take(num_samples_to_visualize):
    sample_images.append(image_batch.numpy())
    sample_gt_bboxes.append(gt_bbox_batch.numpy())

for idx in range(num_samples_to_visualize):
    image = sample_images[idx]
    gt_boxes_for_image = sample_gt_bboxes[idx]

    image_for_prediction = np.expand_dims(image, axis=0)

    predicted_box_for_image = model.predict(image_for_prediction,
        verbose=0)

    predicted_box_for_image = predicted_box_for_image.squeeze(axis=0)

    if predicted_box_for_image.ndim == 1:
        predicted_box_for_image =
np.expand_dims(predicted_box_for_image, axis=0)

    all_boxes = list(gt_boxes_for_image) +
list(predicted_box_for_image)
    all_colors = ['green'] * len(gt_boxes_for_image) + ['red'] *
len(predicted_box_for_image)

    if all_boxes:
        fig, ax = plt.subplots(1, figsize=(8, 8))
        ax.imshow(image)
        for k, box in enumerate(all_boxes):
            xmin, ymin, xmax, ymax = box[0]*TARGET_IMAGE_SIZE[1],
box[1]*TARGET_IMAGE_SIZE[0], box[2]*TARGET_IMAGE_SIZE[1],
box[3]*TARGET_IMAGE_SIZE[0]
            rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax -
ymin, linewidth=2, edgecolor=all_colors[k], facecolor='none')
            ax.add_patch(rect)
            ax.set_title(f'Sample {idx + 1}: GT (Green) & Pred (Red)')
            ax.axis('off')
        plt.show()
    else:

        fig, ax = plt.subplots(1, figsize=(8, 8))
        ax.imshow(image)
        ax.set_title(f'Sample {idx + 1}: No GT BBoxes. Predicted
(Red)')
        for box in predicted_box_for_image:
            xmin, ymin, xmax, ymax = box[0]*TARGET_IMAGE_SIZE[1],
box[1]*TARGET_IMAGE_SIZE[0], box[2]*TARGET_IMAGE_SIZE[1],
box[3]*TARGET_IMAGE_SIZE[0]

```



```
rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax -  
ymin, linewidth=2, edgecolor='red', facecolor='none')  
ax.add_patch(rect)  
ax.axis('off')  
plt.show()
```

Sample 1: GT (Green) & Pred (Red)



Sample 2: GT (Green) & Pred (Red)



Sample 3: GT (Green) & Pred (Red)



Sample 4: GT (Green) & Pred (Red)



Sample 5: GT (Green) & Pred (Red)



```
import pickle

model_filename = 'number_plate_model.pkl'

with open(model_filename, 'wb') as file:
    pickle.dump(model, file)

print(f"Model successfully pickled and saved as '{model_filename}'")
Model successfully pickled and saved as 'number_plate_model.pkl'
```

```

import numpy as np

# Print model summary to show parameter details
model.summary()

# Extract parameter counts programmatically
total_parameters = model.count_params()

trainable_parameters = np.sum([np.prod(v.shape) for v in
model.trainable_variables])
non_trainable_parameters = np.sum([np.prod(v.shape) for v in
model.non_trainable_variables])

# Calculate model size
# Assuming each parameter is a 32-bit float (4 bytes)
bytes_per_parameter = 4
total_size_bytes = total_parameters * bytes_per_parameter
total_size_kb = total_size_bytes / 1024
total_size_mb = total_size_kb / 1024

print(f"\n--- Model Parameter and Size Summary ---")
print(f"Total Parameters: {total_parameters:,}")
print(f"Trainable Parameters: {trainable_parameters:,}")
print(f"Non-Trainable Parameters: {non_trainable_parameters:,}")
print(f"Model Size (Bytes): {total_size_bytes:,}")
print(f"Model Size (KB): {total_size_kb:.2f}")
print(f"Model Size (MB): {total_size_mb:.2f}")
print(f"-----")

Model: "functional_1"

```

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D) input_layer_1[0]...	(None, 112, 112, 32)	864	

bn_Conv1	(None, 112, 112,	128	Conv1[0][0]
(BatchNormalizatio...	32)		
Conv1_relu (ReLU)	(None, 112, 112,	0	bn_Conv1[0]
[0]	32)		
expanded_conv_dept...	(None, 112, 112,	288	Conv1_relu[0]
[0]	(DepthwiseConv2D)	32)	
expanded_conv_dept...	(None, 112, 112,	128	
expanded_conv_de...	(BatchNormalizatio...	32)	
expanded_conv_dept...	(None, 112, 112,	0	
expanded_conv_de...	(ReLU)	32)	
expanded_conv_proj...	(None, 112, 112,	512	
expanded_conv_de...	(Conv2D)	16)	
expanded_conv_proj...	(None, 112, 112,	64	
expanded_conv_pr...	(BatchNormalizatio...	16)	
block_1_expand	(None, 112, 112,	1,536	
expanded_conv_pr...	(Conv2D)	96)	
block_1_expand_BN	(None, 112, 112,	384	

block_1_expand[0... (BatchNormalizatio...	96)		
block_1_expand_relu block_1_expand_B...	(None, 112, 112, 0 (ReLU) 96)		
block_1_pad block_1_expand_r...	(None, 113, 113, 0 (ZeroPadding2D) 96)		
block_1_depthwise block_1_pad[0][0]	(None, 56, 56, 864 (DepthwiseConv2D) 96)		
block_1_depthwise_... block_1_depthwis...	(None, 56, 56, 384 (BatchNormalizatio... 96)		
block_1_depthwise_... block_1_depthwis...	(None, 56, 56, 0 (ReLU) 96)		
block_1_project block_1_depthwis...	(None, 56, 56, 2,304 (Conv2D) 24)		
block_1_project_BN block_1_project[...	(None, 56, 56, 96 (BatchNormalizatio... 24)		
block_2_expand block_1_project_...	(None, 56, 56, 3,456		

(Conv2D)	144)		
block_2_expand_BN block_2_expand[0...	(None, 56, 56, 144)	576	
(BatchNormalizatio...	144)		
block_2_expand_relu block_2_expand_B...	(None, 56, 56, 144)	0	
(ReLU)	144)		
block_2_depthwise block_2_expand_r...	(None, 56, 56, 144)	1,296	
(DepthwiseConv2D)	144)		
block_2_depthwise_... block_2_depthwis...	(None, 56, 56, 144)	576	
(BatchNormalizatio...	144)		
block_2_depthwise_... block_2_depthwis...	(None, 56, 56, 144)	0	
(ReLU)	144)		
block_2_project block_2_depthwis...	(None, 56, 56, 24)	3,456	
(Conv2D)	24)		
block_2_project_BN block_2_project[...	(None, 56, 56, 24)	96	
(BatchNormalizatio...	24)		
block_2_add (Add) block_1_project_...	(None, 56, 56, 24)	0	
	24)		

block_2_project_...			
block_3_expand	(None, 56, 56,	3,456	
block_2_add[0][0]	(Conv2D)	144)	
block_3_expand_BN	(None, 56, 56,	576	
block_3_expand[0...	(BatchNormalizatio...	144)	
block_3_expand_relu	(None, 56, 56,	0	
block_3_expand_B...	(ReLU)	144)	
block_3_pad	(None, 57, 57,	0	
block_3_expand_r...	(ZeroPadding2D)	144)	
block_3_depthwise	(None, 28, 28,	1,296	
block_3_pad[0][0]	(DepthwiseConv2D)	144)	
block_3_depthwise_...	(None, 28, 28,	576	
block_3_depthwis...	(BatchNormalizatio...	144)	
block_3_depthwise_...	(None, 28, 28,	0	
block_3_depthwis...	(ReLU)	144)	
block_3_project	(None, 28, 28,	4,608	
block_3_depthwis...	(Conv2D)	32)	

block_3_project_BN	(None, 28, 28,	128	
block_3_project[...	(BatchNormalizatio...	32)	
block_4_expand	(None, 28, 28,	6,144	
block_3_project_...	(Conv2D)	192)	
block_4_expand_BN	(None, 28, 28,	768	
block_4_expand[0...	(BatchNormalizatio...	192)	
block_4_expand_relu	(None, 28, 28,	0	
block_4_expand_B...	(ReLU)	192)	
block_4_depthwise	(None, 28, 28,	1,728	
block_4_expand_r...	(DepthwiseConv2D)	192)	
block_4_depthwise_...	(None, 28, 28,	768	
block_4_depthwis...	(BatchNormalizatio...	192)	
block_4_depthwise_...	(None, 28, 28,	0	
block_4_depthwis...	(ReLU)	192)	
block_4_project	(None, 28, 28,	6,144	
block_4_depthwis...	(Conv2D)	32)	

block_4_project_BN	(None, 28, 28,	128
block_4_project[...]	(BatchNormalizatio...	32)
block_4_add (Add)	(None, 28, 28,	0
block_3_project_...	32)	
block_4_project_...		
block_5_expand	(None, 28, 28,	6,144
block_4_add[0][0]	(Conv2D)	192)
block_5_expand_BN	(None, 28, 28,	768
block_5_expand[0...]	(BatchNormalizatio...	192)
block_5_expand_relu	(None, 28, 28,	0
block_5_expand_B...	(ReLU)	192)
block_5_depthwise	(None, 28, 28,	1,728
block_5_expand_r...	(DepthwiseConv2D)	192)
block_5_depthwise_...	(None, 28, 28,	768
block_5_depthwis...	(BatchNormalizatio...	192)
block_5_depthwise_...	(None, 28, 28,	0
block_5_depthwis...	(ReLU)	192)

block_5_project block_5_depthwis...	(None, 28, 28, 32)	6,144	
(Conv2D)			
block_5_project_BN block_5_project[...	(None, 28, 28, 32)	128	
(BatchNormalizatio...			
block_5_add (Add) block_4_add[0][0...	(None, 28, 28, 32)	0	
block_5_project_...			
block_6_expand block_5_add[0][0]	(None, 28, 28, 192)	6,144	
(Conv2D)			
block_6_expand_BN block_6_expand[0...	(None, 28, 28, 192)	768	
(BatchNormalizatio...			
block_6_expand_relu block_6_expand_B...	(None, 28, 28, 192)	0	
(ReLU)			
block_6_pad block_6_expand_r...	(None, 29, 29, 192)	0	
(ZeroPadding2D)			
block_6_depthwise block_6_pad[0][0]	(None, 14, 14, 192)	1,728	
(DepthwiseConv2D)			
block_6_depthwise_...	(None, 14, 14, 768)		

block_6_depthwise...	(BatchNormalizatio...	192)		
block_6_depthwise_...	(None, 14, 14,	0		
block_6_depthwis...	(ReLU)	192)		
block_6_project	(None, 14, 14,	12,288		
block_6_depthwis...	(Conv2D)	64)		
block_6_project_BN	(None, 14, 14,	256		
block_6_project[...	(BatchNormalizatio...	64)		
block_7_expand	(None, 14, 14,	24,576		
block_6_project_...	(Conv2D)	384)		
block_7_expand_BN	(None, 14, 14,	1,536		
block_7_expand[0...	(BatchNormalizatio...	384)		
block_7_expand_relu	(None, 14, 14,	0		
block_7_expand_B...	(ReLU)	384)		
block_7_depthwise	(None, 14, 14,	3,456		
block_7_expand_r...	(DepthwiseConv2D)	384)		
block_7_depthwise_...	(None, 14, 14,	1,536		
block_7_depthwis...				

(BatchNormalizatio...	384)		
block_7_depthwise_...	(None, 14, 14,	0	
block_7_depthwis...	(ReLU)	384)	
block_7_project	(None, 14, 14,	24,576	
block_7_depthwis...	(Conv2D)	64)	
block_7_project_BN	(None, 14, 14,	256	
block_7_project[...	(BatchNormalizatio...	64)	
block_7_add (Add)	(None, 14, 14,	0	
block_6_project_...		64)	
block_7_project_...			
block_8_expand	(None, 14, 14,	24,576	
block_7_add[0][0]	(Conv2D)	384)	
block_8_expand_BN	(None, 14, 14,	1,536	
block_8_expand[0...	(BatchNormalizatio...	384)	
block_8_expand_relu	(None, 14, 14,	0	
block_8_expand_B...	(ReLU)	384)	
block_8_depthwise	(None, 14, 14,	3,456	
block_8_expand_r...	(DepthwiseConv2D)	384)	

block_8_depthwise_...	(None, 14, 14,	1,536	
block_8_depthwis...	(BatchNormalizatio...	384)	
block_8_depthwise_...	(None, 14, 14,	0	
block_8_depthwis...	(ReLU)	384)	
block_8_project	(None, 14, 14,	24,576	
block_8_depthwis...	(Conv2D)	64)	
block_8_project_BN	(None, 14, 14,	256	
block_8_project[...	(BatchNormalizatio...	64)	
block_8_add (Add)	(None, 14, 14,	0	
block_7_add[0][0...			
	64)		
block_8_project_...			
block_9_expand	(None, 14, 14,	24,576	
block_8_add[0][0]	(Conv2D)	384)	
block_9_expand_BN	(None, 14, 14,	1,536	
block_9_expand[0...	(BatchNormalizatio...	384)	
block_9_expand_relu	(None, 14, 14,	0	
block_9_expand_B...	(ReLU)	384)	

block_9_depthwise block_9_expand_r...	(None, 14, 14,	3,456	
(DepthwiseConv2D)	384)		
block_9_depthwise_...	(None, 14, 14,	1,536	
block_9_depthwis...	(BatchNormalizatio...	384)	
block_9_depthwise_...	(None, 14, 14,	0	
block_9_depthwis...	(ReLU)	384)	
block_9_project	(None, 14, 14,	24,576	
block_9_depthwis...	(Conv2D)	64)	
block_9_project_BN	(None, 14, 14,	256	
block_9_project[...	(BatchNormalizatio...	64)	
block_9_add (Add)	(None, 14, 14,	0	
block_8_add[0][0...	64)		
block_9_project_...			
block_10_expand	(None, 14, 14,	24,576	
block_9_add[0][0]	(Conv2D)	384)	
block_10_expand_BN	(None, 14, 14,	1,536	
block_10_expand[...	(BatchNormalizatio...	384)	

block_10_expand_relu	(None, 14, 14,	0
block_10_expand_relu (ReLU)	384)	
block_10_depthwise	(None, 14, 14,	3,456
block_10_depthwise (DepthwiseConv2D)	384)	
block_10_depthwise	(None, 14, 14,	1,536
block_10_depthwise (BatchNormalization)	384)	
block_10_depthwise	(None, 14, 14,	0
block_10_depthwise (ReLU)	384)	
block_10_project	(None, 14, 14,	36,864
block_10_project (Conv2D)	96)	
block_10_project_BN	(None, 14, 14,	384
block_10_project_BN (BatchNormalization)	96)	
block_11_expand	(None, 14, 14,	55,296
block_11_expand (Conv2D)	576)	
block_11_expand_BN	(None, 14, 14,	2,304
block_11_expand_BN (BatchNormalization)	576)	
block_11_expand_relu	(None, 14, 14,	0

block_11_expand_...	(ReLU)	576)		
block_11_depthwise	(None, 14, 14,	5,184		
block_11_expand_...	(DepthwiseConv2D)	576)		
block_11_depthwise...	(None, 14, 14,	2,304		
block_11_depthwi...	(BatchNormalizatio...	576)		
block_11_depthwise...	(None, 14, 14,	0		
block_11_depthwi...	(ReLU)	576)		
block_11_project	(None, 14, 14,	55,296		
block_11_depthwi...	(Conv2D)	96)		
block_11_project_BN	(None, 14, 14,	384		
block_11_project...	(BatchNormalizatio...	96)		
block_11_add (Add)	(None, 14, 14,	0		
block_10_project...		96)		
block_11_project...				
block_12_expand	(None, 14, 14,	55,296		
block_11_add[0][...	(Conv2D)	576)		
block_12_expand_BN	(None, 14, 14,	2,304		
block_12_expand[...				

(BatchNormalizatio...	576)		
block_12_expand_re...	(None, 14, 14,	0	
block_12_expand_...	(ReLU)	576)	
block_12_depthwise	(None, 14, 14,	5,184	
block_12_expand_...	(DepthwiseConv2D)	576)	
block_12_depthwise...	(None, 14, 14,	2,304	
block_12_depthwi...	(BatchNormalizatio...	576)	
block_12_depthwise...	(None, 14, 14,	0	
block_12_depthwi...	(ReLU)	576)	
block_12_project	(None, 14, 14,	55,296	
block_12_depthwi...	(Conv2D)	96)	
block_12_project_BN	(None, 14, 14,	384	
block_12_project...	(BatchNormalizatio...	96)	
block_12_add (Add)	(None, 14, 14,	0	
block_11_add[0][...		96)	
block_12_project...			
block_13_expand	(None, 14, 14,	55,296	
block_12_add[0][...	(Conv2D)	576)	

block_13_expand_BN	(None, 14, 14,	2,304	
block_13_expand[...] (BatchNormalizatio...	576)		
block_13_expand_re...	(None, 14, 14,	0	
block_13_expand[...] (ReLU)	576)		
block_13_pad	(None, 15, 15,	0	
block_13_expand[...] (ZeroPadding2D)	576)		
block_13_depthwise	(None, 7, 7, 576)	5,184	
block_13_pad[0][...] (DepthwiseConv2D)			
block_13_depthwise...	(None, 7, 7, 576)	2,304	
block_13_depthwi... (BatchNormalizatio...			
block_13_depthwise...	(None, 7, 7, 576)	0	
block_13_depthwi... (ReLU)			
block_13_project	(None, 7, 7, 160)	92,160	
block_13_depthwi... (Conv2D)			
block_13_project_BN	(None, 7, 7, 160)	640	
block_13_project... (BatchNormalizatio...			

block_14_expand	(None, 7, 7, 960)	153,600	
block_13_project...			
(Conv2D)			
block_14_expand_BN	(None, 7, 7, 960)	3,840	
block_14_expand[...			
(BatchNormalizatio...			
block_14_expand_re...	(None, 7, 7, 960)	0	
block_14_expand_...			
(ReLU)			
block_14_depthwise	(None, 7, 7, 960)	8,640	
block_14_expand_...			
(DepthwiseConv2D)			
block_14_depthwise...	(None, 7, 7, 960)	3,840	
block_14_depthwi...			
(BatchNormalizatio...			
block_14_depthwise...	(None, 7, 7, 960)	0	
block_14_depthwi...			
(ReLU)			
block_14_project	(None, 7, 7, 160)	153,600	
block_14_depthwi...			
(Conv2D)			
block_14_project_BN	(None, 7, 7, 160)	640	
block_14_project...			
(BatchNormalizatio...			

block_14_add (Add)	(None, 7, 7, 160)	0
block_13_project...		
block_14_project...		
block_15_expand	(None, 7, 7, 960)	153,600
block_14_add[0][... (Conv2D)		
block_15_expand_BN	(None, 7, 7, 960)	3,840
block_15_expand[... (BatchNormalizatio...		
block_15_expand_re...	(None, 7, 7, 960)	0
block_15_expand_... (ReLU)		
block_15_depthwise	(None, 7, 7, 960)	8,640
block_15_expand_... (DepthwiseConv2D)		
block_15_depthwise...	(None, 7, 7, 960)	3,840
block_15_depthwi... (BatchNormalizatio...		
block_15_depthwise...	(None, 7, 7, 960)	0
block_15_depthwi... (ReLU)		
block_15_project	(None, 7, 7, 160)	153,600
block_15_depthwi... (Conv2D)		

block_15_project_BN	(None, 7, 7, 160)	640	
block_15_project...			
(BatchNormalizatio...			
block_15_add (Add)	(None, 7, 7, 160)	0	
block_14_add[0][...			
block_15_project...			
block_16_expand	(None, 7, 7, 960)	153,600	
block_15_add[0][...			
(Conv2D)			
block_16_expand_BN	(None, 7, 7, 960)	3,840	
block_16_expand[...			
(BatchNormalizatio...			
block_16_expand_re...	(None, 7, 7, 960)	0	
block_16_expand_...			
(ReLU)			
block_16_depthwise	(None, 7, 7, 960)	8,640	
block_16_expand_...			
(DepthwiseConv2D)			
block_16_depthwise...	(None, 7, 7, 960)	3,840	
block_16_depthwi...			
(BatchNormalizatio...			
block_16_depthwise...	(None, 7, 7, 960)	0	
block_16_depthwi...			
(ReLU)			
block_16_project	(None, 7, 7, 320)	307,200	

block_16_depthwi...	(Conv2D)			
block_16_project_BN	(None, 7, 7, 320)	1,280		
block_16_project...	(BatchNormalizatio...			
Conv_1 (Conv2D)	(None, 7, 7,	409,600		
block_16_project...	1280)			
Conv_1_bn	(None, 7, 7,	5,120	Conv_1[0][0]	
(BatchNormalizatio...	1280)			
out_relu (ReLU)	(None, 7, 7,	0	Conv_1_bn[0]	
[0]	1280)			
global_average_poo...	(None, 1280)	0	out_relu[0]	
[0]	(GlobalAveragePool...			
dense (Dense)	(None, 128)	163,968		
global_average_p...				
dense_1 (Dense)	(None, 64)	8,256	dense[0][0]	
bbox_output (Dense)	(None, 4)	260	dense_1[0][0]	

Total params: 2,775,438 (10.59 MB)

```
Trainable params: 172,484 (673.77 KB)
Non-trainable params: 2,257,984 (8.61 MB)
Optimizer params: 344,970 (1.32 MB)
```

```
--- Model Parameter and Size Summary ---
Total Parameters: 2,430,468
Trainable Parameters: 172,484
Non-Trainable Parameters: 2,257,984
Model Size (Bytes): 9,721,872
Model Size (KB): 9494.02
Model Size (MB): 9.27
-----
```

Task

Quantize the trained Keras model using TensorFlow Lite with float16 post-training quantization to reduce its size, then save the quantized model to a file.

Quantize the Model

Subtask:

Convert the trained Keras model into a TensorFlow Lite model with float16 post-training quantization to reduce its size, and save it.

Reasoning: I need to convert the trained Keras model into a TensorFlow Lite model with float16 post-training quantization and save it, following the provided instructions.

```
import tensorflow as tf

converter = tf.lite.TFLiteConverter.from_keras_model(model)

converter.optimizations = [tf.lite.Optimize.DEFAULT]

converter.target_spec.supported_types = [tf.float16]

tflite_model = converter.convert()
```

```
with open('quantized_model_float16.tflite', 'wb') as f:  
    f.write(tflite_model)
```

```
print("Keras model successfully converted to float16 quantized TFLite  
model and saved as 'quantized_model_float16.tflite'")
```

Saved artifact at '/tmp/tmp2qjixduv'. The following endpoints are available:

* Endpoint 'serve'

args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3),
dtype=tf.float32, name='keras_tensor_11')

Output Type:

TensorSpec(shape=(None, 4), dtype=tf.float32, name=None)

Captures:

139298275919440: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275920976: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275920784: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275919056: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275921552: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275921936: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275922128: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275921168: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275921744: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275922704: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275923088: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275923280: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275922320: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275922896: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275923856: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275921360: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275923472: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275923664: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275920592: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275924816: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275924240: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275925968: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275926160: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275925392: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275922512: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275925008: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275924624: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275924432: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275925584: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275927120: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275926544: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275926736: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275926928: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298275925776: TensorSpec(shape=(), dtype=tf.resource, name=None)

[illegible]

[illegible]

[illegible]

[illegible]

```
139298273458320: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458512: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458704: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273456208: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459856: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459280: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458128: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459664: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459472: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459088: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457168: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216464: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216656: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273460048: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215504: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215120: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216272: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216080: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215888: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217616: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217040: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217232: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217424: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215312: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218576: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218000: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218192: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218384: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272219536: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298742888656: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216848: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272220496: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272219728: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272220880: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272221072: TensorSpec(shape=(), dtype=tf.resource, name=None)
Keras model successfully converted to float16 quantized TFLite model
and saved as 'quantized_model_float16.tflite'
```

```
import tensorflow as tf
import numpy as np
```

```
interpreter =
tf.lite.Interpreter(model_path='quantized_model_float16.tflite')
```

```
interpreter.allocate_tensors()
```

```

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("Quantized TFLite model loaded successfully.")
print("Input details:", input_details)
print("Output details:", output_details)

Quantized TFLite model loaded successfully.
Input details: [{'name': 'serving_default_keras_tensor_11:0', 'index': 0, 'shape': array([ 1, 224, 224,  3], dtype=int32), 'shape_signature': array([-1, 224, 224,  3], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([], dtype=float32), 'zero_points': array([], dtype=int32), 'quantized_dimension': 0}, 'sparsity_parameters': {}}]
Output details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 288, 'shape': array([1, 4], dtype=int32), 'shape_signature': array([-1, 4], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization': (0.0, 0), 'quantization_parameters': {'scales': array([], dtype=float32), 'zero_points': array([], dtype=int32), 'quantized_dimension': 0}, 'sparsity_parameters': {}}]

/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning: Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
  See the [migration guide](https://ai.google.dev/edge/litert/migration) for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

import tensorflow as tf
import numpy as np

interpreter =
tf.lite.Interpreter(model_path='quantized_model_float16.tflite')

interpreter.allocate_tensors()

input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

print("Quantized TFLite model loaded successfully.")
print("Input details:", input_details)
print("Output details:", output_details)

total_val_loss = 0.0

```

```

num_batches = 0

for images_batch, gt_bboxes_batch in val_dataset:
    num_batches += 1
    batch_predicted_bboxes = []

    for i in tf.range(tf.shape(images_batch)[0]):
        input_image = images_batch[i]

        input_image = tf.expand_dims(input_image, axis=0)

        input_image = tf.cast(input_image, input_details[0]['dtype'])

        interpreter.set_tensor(input_details[0]['index'], input_image)
        interpreter.invoke()
        output_data = interpreter.get_tensor(output_details[0]
['index'])

        batch_predicted_bboxes.append(output_data.squeeze(axis=0))

    batch_predicted_bboxes_tensor = tf.stack(batch_predicted_bboxes,
axis=0)

    current_batch_loss = bbox_loss(gt_bboxes_batch,
batch_predicted_bboxes_tensor)
    total_val_loss += current_batch_loss

avg_val_loss = total_val_loss / num_batches

print(f"\nValidation Loss of Quantized TFLite Model:
{avg_val_loss:.4f}")

/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/
interpreter.py:457: UserWarning:      Warning: tf.lite.Interpreter is
deprecated and is scheduled for deletion in
TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert
package.
  See the [migration
guide](https://ai.google.dev/edge/litert/migration)
  for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

Quantized TFLite model loaded successfully.
Input details: [{'name': 'serving_default_keras_tensor_11:0', 'index':

```

```
0, 'shape': array([ 1, 224, 224,  3], dtype=int32),
'shape_signature': array([-1, 224, 224,  3], dtype=int32), 'dtype':
<class 'numpy.float32'>, 'quantization': (0.0, 0),
'quantization_parameters': {'scales': array([], dtype=float32),
'zero_points': array([], dtype=int32), 'quantized_dimension': 0},
'sparsity_parameters': {}}]
Output details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 288,
'shape': array([1, 4], dtype=int32), 'shape_signature': array([-1,
4], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization':
(0.0, 0), 'quantization_parameters': {'scales': array([],
dtype=float32), 'zero_points': array([], dtype=int32),
'quantized_dimension': 0}, 'sparsity_parameters': {}}]
```

Validation Loss of Quantized TFLite Model: 0.0042

```
import tensorflow as tf
import numpy as np
```

```
interpreter =
tf.lite.Interpreter(model_path='quantized_model_float16.tflite')
```

```
interpreter.allocate_tensors()
```

```
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()
```

```
print("Quantized TFLite model loaded successfully.")
print("Input details:", input_details)
print("Output details:", output_details)
```

```
total_val_loss = 0.0
num_batches = 0
```

```
for images_batch, gt_bboxes_batch in val_dataset:
    num_batches += 1
    batch_predicted_bboxes = []
```

```
    for i in tf.range(tf.shape(images_batch)[0]):
        input_image = images_batch[i]
```

```
        input_image = tf.expand_dims(input_image, axis=0)
```

```
        input_image = tf.cast(input_image, input_details[0]['dtype'])
```



```

        interpreter.set_tensor(input_details[0]['index'], input_image)
        interpreter.invoke()
        output_data = interpreter.get_tensor(output_details[0]
['index'])

        batch_predicted_bboxes.append(output_data.squeeze(axis=0))

    batch_predicted_bboxes_tensor = tf.stack(batch_predicted_bboxes,
axis=0)

    current_batch_loss = bbox_loss(gt_bboxes_batch,
batch_predicted_bboxes_tensor)
    total_val_loss += current_batch_loss

avg_val_loss = total_val_loss / num_batches

print(f"\nValidation Loss of Quantized TFLite Model:
{avg_val_loss:.4f}")

/usr/local/lib/python3.12/dist-packages/tensorflow/lite/python/
interpreter.py:457: UserWarning:      Warning: tf.lite.Interpreter is
deprecated and is scheduled for deletion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert
package.
    See the [migration
guide](https://ai.google.dev/edge/litert/migration)
    for details.

warnings.warn(_INTERPRETER_DELETION_WARNING)

Quantized TFLite model loaded successfully.
Input details: [{'name': 'serving_default_keras_tensor_11:0', 'index':
0, 'shape': array([ 1, 224, 224,   3], dtype=int32),
'shape_signature': array([-1, 224, 224,   3], dtype=int32), 'dtype':
<class 'numpy.float32'>, 'quantization': (0.0, 0),
'quantization_parameters': {'scales': array([], dtype=float32),
'zero_points': array([], dtype=int32), 'quantized_dimension': 0},
'sparsity_parameters': {}}]
Output details: [{'name': 'StatefulPartitionedCall_1:0', 'index': 288,
'shape': array([1, 4], dtype=int32), 'shape_signature': array([-1,
4], dtype=int32), 'dtype': <class 'numpy.float32'>, 'quantization':
(0.0, 0), 'quantization_parameters': {'scales': array([],
dtype=float32), 'zero_points': array([], dtype=int32),
'quantized_dimension': 0}, 'sparsity_parameters': {}}]

Validation Loss of Quantized TFLite Model: 0.0023

num_samples_to_visualize = 5

```

```

sample_images = []
sample_gt_bboxes = []

for image_batch, gt_bbox_batch in
val_dataset.unbatch().take(num_samples_to_visualize):
    sample_images.append(image_batch.numpy())
    sample_gt_bboxes.append(gt_bbox_batch.numpy())

for idx in range(num_samples_to_visualize):
    image = sample_images[idx]
    gt_boxes_for_image = sample_gt_bboxes[idx]

    image_for_prediction = np.expand_dims(image, axis=0)
    input_tensor = tf.cast(image_for_prediction, input_details[0]
['dtype'])

    interpreter.set_tensor(input_details[0]['index'], input_tensor)

    interpreter.invoke()

    predicted_box_for_image = interpreter.get_tensor(output_details[0]
['index'])

    predicted_box_for_image = predicted_box_for_image.squeeze(axis=0)

    if predicted_box_for_image.ndim == 1:
        predicted_box_for_image =
np.expand_dims(predicted_box_for_image, axis=0)

    all_boxes = list(gt_boxes_for_image) +
list(predicted_box_for_image)
    all_colors = ['green'] * len(gt_boxes_for_image) + ['red'] *
len(predicted_box_for_image)

    if all_boxes:
        fig, ax = plt.subplots(1, figsize=(8, 8))
        ax.imshow(image)
        for k, box in enumerate(all_boxes):
            xmin, ymin, xmax, ymax = box[0]*TARGET_IMAGE_SIZE[1],
box[1]*TARGET_IMAGE_SIZE[0], box[2]*TARGET_IMAGE_SIZE[1],
box[3]*TARGET_IMAGE_SIZE[0]
            rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax -
ymin, linewidth=2, edgecolor=all_colors[k], facecolor='none')
            ax.add_patch(rect)
            ax.set_title(f'Quantized Model Sample {idx + 1}: GT (Green) &

```

```

Pred (Red)')
    ax.axis('off')
    plt.show()
else:

    fig, ax = plt.subplots(1, figsize=(8, 8))
    ax.imshow(image)
    ax.set_title(f'Quantized Model Sample {idx + 1}: No GT BBoxes.
Predicted (Red)')
    for box in predicted_box_for_image:
        xmin, ymin, xmax, ymax = box[0]*TARGET_IMAGE_SIZE[1],
box[1]*TARGET_IMAGE_SIZE[0], box[2]*TARGET_IMAGE_SIZE[1],
box[3]*TARGET_IMAGE_SIZE[0]
        rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax -
ymin, linewidth=2, edgecolor='red', facecolor='none')
        ax.add_patch(rect)
    ax.axis('off')
    plt.show()

print("Visualization of quantized model predictions completed.")

```

Quantized Model Sample 1: GT (Green) & Pred (Red)



Quantized Model Sample 2: GT (Green) & Pred (Red)



Quantized Model Sample 3: GT (Green) & Pred (Red)



Quantized Model Sample 4: GT (Green) & Pred (Red)



Quantized Model Sample 5: GT (Green) & Pred (Red)



Visualization of quantized model predictions completed.

```
import os
```

```
original_model_path = 'number_plate_model.pkl'  
quantized_model_path = 'quantized_model_float16.tflite'
```

```
original_size = os.path.getsize(original_model_path)  
quantized_size = os.path.getsize(quantized_model_path)
```

```
print(f"Original Model Size: {original_size / (1024 * 1024):.2f} MB")  
print(f"Quantized Model (Float16) Size: {quantized_size / (1024 *
```

```
1024):.2f} MB")
```

```
size_reduction_percent = ((original_size - quantized_size) /  
original_size) * 100  
print(f"Size Reduction: {size_reduction_percent:.2f}%")
```

```
Original Model Size: 11.15 MB  
Quantized Model (Float16) Size: 4.58 MB  
Size Reduction: 58.91%
```

```
model.export('number_plate_keras_model')  
print("Keras model saved successfully as 'number_plate_keras_model' in  
TensorFlow SavedModel format.")
```

Saved artifact at 'number_plate_keras_model'. The following endpoints are available:

* Endpoint 'serve'

```
args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3),  
dtype=tf.float32, name='keras_tensor_11')
```

Output Type:

```
TensorSpec(shape=(None, 4), dtype=tf.float32, name=None)
```

Captures:

```
139298275919440: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275920976: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275920784: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275919056: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275921552: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275921936: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275922128: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275921168: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275921744: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275922704: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275923088: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275923280: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275922320: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275922896: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275923856: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275921360: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275923472: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275923664: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275920592: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275924816: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275924240: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275925968: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275926160: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275925392: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275922512: TensorSpec(shape=(), dtype=tf.resource, name=None)  
139298275925008: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

[illegible]

[illegible]

[illegible]

[illegible]

```
139298273456784: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273454288: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457936: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457360: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457552: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457744: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273455248: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458896: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458320: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458512: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458704: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273456208: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459856: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459280: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273458128: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459664: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459472: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273459088: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273457168: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216464: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216656: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298273460048: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215504: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215120: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216272: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216080: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215888: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217616: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217040: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217232: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272217424: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215312: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218576: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218000: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218192: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272218384: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272215696: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272219536: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298742888656: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272216848: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272220496: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272219728: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272220880: TensorSpec(shape=(), dtype=tf.resource, name=None)
139298272221072: TensorSpec(shape=(), dtype=tf.resource, name=None)
Keras model saved successfully as 'number_plate_keras_model' in
TensorFlow SavedModel format.
```